

Pazimo - Technical Documentation

Table of Contents

1. [Project Overview](#)
 2. [Tech Stack](#)
 3. [Architecture](#)
 4. [Database Models](#)
 5. [API Documentation](#)
 6. [Frontend Routes](#)
 7. [Key Features](#)
 8. [Development Setup](#)
 9. [Deployment](#)
 10. [Security Considerations](#)
-

Project Overview

Pazimo is a comprehensive event ticketing and RSVP platform built for the Ethiopian market. The platform enables event organizers to create events, sell tickets, manage invitations, and collect RSVPs through customizable forms. It supports multiple payment gateways (Chapa, Santimpay), QR code ticketing, and real-time analytics.

Key Capabilities

- Event creation and management with multiple ticket types
- Online ticket sales with payment gateway integration
- QR code-based ticket validation and check-in
- Invitation system with email/SMS delivery

- Customizable RSVP forms with multiple question types
 - Organizer revenue management and withdrawals
 - Admin dashboard for platform oversight
 - Multi-currency support (ETB, USD)
 - Real-time analytics and reporting
-

Tech Stack

Frontend

- **Framework:** Next.js 15.3.6 (App Router)
- **Language:** TypeScript
- **UI Components:** Radix UI primitives
- **Styling:** Tailwind CSS 4
- **State Management:** Zustand
- **Forms:** React DatePicker, custom form components
- **QR Code:** @yudiel/react-qr-scanner, html5-qrcode, qrcode.react
- **Charts:** Recharts
- **HTTP Client:** Axios
- **Real-time:** Socket.IO Client
- **Animations:** Framer Motion
- **Utilities:** date-fns, clsx, tailwind-merge

Backend

- **Runtime:** Node.js
- **Framework:** Express.js
- **Database:** MongoDB with Mongoose
- **Authentication:** JWT (jsonwebtoken), bcryptjs

- **File Upload:** Multer
- **Cloud Storage:** Cloudinary
- **Real-time:** Socket.IO
- **Payment Gateways:**
 - Chapa (chapa-nodejs, @chapa_et/inline.js)
 - Santimpay (custom SDK)
- **Email:** Nodemailer
- **SMS:** Custom integration
- **Security:** Helmet, CORS
- **Logging:** Morgan

Development Tools

- **Frontend Build:** Next.js with Turbopack
 - **Backend Development:** Nodemon
 - **Code Quality:** ESLint
 - **Version Control:** Git
-

Architecture

Project Structure

```
pazimo/
├── frontend/                # Next.js frontend application
│   ├── app/                # Next.js App Router pages
│   │   ├── (auth)/         # Authentication pages
│   │   ├── admin/          # Admin dashboard
│   │   ├── organizer/      # Organizer dashboard
│   │   ├── my-account/     # User account pages
│   │   └── page.tsx         # Landing page
│   ├── components/         # Reusable React components
│   ├── lib/                # Utility functions
│   ├── hooks/              # Custom React hooks
│   ├── store/              # Zustand state management
│   ├── types/              # TypeScript type definitions
│   └── utils/              # Utility functions
├── backend/                # Express.js backend API
│   ├── src/
│   │   ├── config/         # Database and server configuration
│   │   ├── controllers/    # Business logic handlers
│   │   ├── middleware/     # Custom middleware
│   │   ├── models/         # Mongoose database models
│   │   ├── routes/         # API route definitions
│   │   ├── services/       # External service integrations
│   │   ├── utils/          # Utility functions
│   │   ├── scripts/        # Database maintenance scripts
│   │   ├── app.js          # Express application setup
│   │   └── server.js       # Server entry point
│   ├── uploads/            # File upload storage
│   └── santimSDK/          # Santimpay payment SDK
└── node_modules/           # Dependencies
```

Architecture Patterns

Frontend Architecture

- **Page-based routing** using Next.js App Router
- **Component composition** with reusable UI components
- **State management** using Zustand stores
- **API communication** through Axios with centralized error handling
- **Real-time updates** via Socket.IO client
- **Code splitting** and lazy loading for performance

Backend Architecture

- **RESTful API design** with Express.js
 - **MVC pattern** with separate controllers, models, and routes
 - **Middleware layer** for authentication, validation, and error handling
 - **Service layer** for external integrations (payment, email, SMS)
 - **Real-time communication** via Socket.IO
 - **Database indexing** for query optimization
-

Database Models

Core Models

User

Represents platform users with role-based access control.

Fields:

- `email` (String, unique, required)
- `phoneNumber` (String, required)
- `password` (String, required, hashed)
- `role` (Enum: customer, organizer, admin, partner)
- `firstName` (String, required)
- `lastName` (String, conditional)

- `isActive` (Boolean)
- `isPhoneVerified` (Boolean)
- `tickets` (Array of Ticket references)
- `wishlist` (Array of Event references)
- `passwordResetToken` , `passwordResetExpires`

Indexes:

- Email uniqueness
- Phone number + role compound index
- Role index
- Full-text search index on name fields

Virtuals:

- `fullName` - Concatenated first and last name
- `organizerProfile` - Organizer registration details

Event

Represents events created by organizers.

Fields:

- `title` (String, required)
- `slug` (String, auto-generated)
- `shortId` (String, unique, auto-generated)
- `isFeatured` , `isTrending` (Boolean)
- `description` (String, required)
- `category` (ObjectId ref: Category)
- `isPublic` , `isSoldOut` (Boolean)
- `startDate` , `endDate` (Date)
- `startTime` , `endTime` (String)
- `location` (GeoJSON object with address, city, country)

- `organizer` (ObjectId ref: User)
- `coverImages` (Array of strings)
- `eventImages` (Array of objects with url and caption)
- `ticketTypes` (Array of ticket type objects with pricing, quantity, wave management)
- `status` (Enum: draft, published, cancelled, completed)
- `bannerStatus` (Boolean)
- `capacity` (Number)
- `tags` (Array of strings)
- `ageRestriction` (Object with min/max age)

Ticket Types Features:

- Multi-currency pricing (ETB, USD)
- Wave-based availability with multiple switch modes
- Date and quantity-based availability rules
- Manual disable capability

Indexes:

- Organizer + created date compound index
- Status + start date compound index
- Category + status compound index
- Short ID unique index

Ticket

Represents purchased or invitation tickets.

Fields:

- `ticketId` (String, unique, auto-generated)
- `isInvitation` , `isOnDoor` (Boolean)
- `event` (ObjectId ref: Event)
- `user` (ObjectId ref: User, conditional)

- `guestName` , `guestEmail` , `guestPhone` (String, conditional for invitations)
- `ticketType` (String)
- `price` (Number)
- `currency` (Enum: ETB, USD)
- `purchaseDate` (Date)
- `status` (Enum: active, used, cancelled, expired, pending, confirmed, declined)
- `paymentStatus` (Enum: pending, completed, failed)
- `paymentDate` , `paymentReference` (String)
- `ticketCount` , `purchaseQuantity` (Number)
- `invitationId` (String, sparse)
- `checkedIn` , `checkedAt` (Boolean/Date)
- `qrCode` (String - auto-generated SVG with logo)
- `message` (String)

QR Code Features:

- Auto-generated on ticket creation
- Circle-style QR dots with rounded finder eyes
- Embedded logo in center
- Encrypted payload with ticket details
- Base64 SVG format

Indexes:

- Compound indexes for event queries with status, payment status, currency
- User-based indexes for customer ticket queries
- Invitation-specific indexes
- Check-in optimization indexes

RsvpForm

Represents customizable RSVP/review forms.

Fields:

- `formId` , `publicId` (String, unique UUID)
- `slug` (String, unique)
- `organizerId` (ObjectId ref: User)
- `title` , `description` (String)
- `type` (Enum: rsvp, review)
- `status` (Enum: draft, published, cancelled, hidden, archived, review, closed)
- `isFeatured` , `isTrending` , `bannerStatus` , `isPublic` (Boolean)
- `coverImage` , `date` , `hostedBy` (String)
- `startTime` , `endTime` (String)
- `location` , `venue` (String)
- `rsvpLimit` (Number)
- `approvalMode` (Enum: auto, manual)
- `collectAttendeeInfo` (Boolean)
- `payment` (Object with enabled, price, currency, deadline)
- `anonymous` (Boolean)
- `sections` (Array of section objects)
- `questions` (Array of question objects with multiple types)
- `responseCount` , `viewCount` (Number)
- Timestamps for status changes

Question Types:

- `short_text`, `long_text`, `single_choice`, `multi_choice`, `dropdown`
- `phone`, `email`, `file`, `date`
- `rating`, `emoji`, `nps`, `yes_no`
- Conditional logic support

Indexes:

- Organizer-based queries

- Status-based filtering
- Feature/trending/banner indexes
- Public ID lookup

Invitation

Represents event invitations sent to guests.

Fields:

- `invitationId` (String, unique UUID)
- `eventId` (ObjectId ref: Event)
- `organizerId` (ObjectId ref: User)
- `guestName` , `guestEmail` , `guestPhone` (String)
- `guestType` (Enum: guest, paid)
- `ticketType` (String)
- `type` (Enum: email, sms, both)
- `amount` (Number)
- `status` (Enum: pending_payment, paid, sent, delivered, failed, confirmed, declined)
- `paymentStatus` , `paymentReference` (String)
- `ticketId` (String, sparse)
- `qrCodeData` , `rsvpLink` (String)
- `rsvpStatus` (Enum: pending, confirmed, declined)
- `rsvpConfirmedAt` (Date)
- `message` (String)

Indexes:

- Organizer and event-based queries
- Status tracking indexes
- Payment status monitoring
- Guest contact lookup

Supporting Models

Admin - Platform administrator accounts

Campaign - Marketing campaigns

CampaignPricing - Campaign-specific pricing tiers

Category - Event categories

InvitationPricing - Invitation pricing tiers

Notification - User notifications

OrganizerRegistration - Organizer profile information

Payment - Payment transaction records

PaymentConfig - Payment gateway configuration

QRTicket - QR code ticket references

SantimTransaction - Santimpay transaction records

ShortUrl - URL shortening service

Wishlist - User event wishlists

Withdrawal - Organizer withdrawal requests

API Documentation

Base URL

- Development: `http://localhost:5000`
- Production: `https://pazimo.com`

Authentication

Most endpoints require JWT authentication via `Authorization: Bearer {token}` header.

Core Endpoints

Authentication (`/api/auth`)

- `POST /register` - User registration

- `POST /login` - User login
- `POST /logout` - User logout
- `POST /forgot-password` - Password reset request
- `POST /reset-password` - Password reset confirmation

Users (`/api/users`)

- `GET /users` - List users (admin only)
- `GET /users/:id` - Get user by ID
- `PUT /users/:id` - Update user profile
- `GET /users/organizers-stats` - Get organizers with statistics (optimized admin endpoint)
- `GET /users/organizer/:id` - Get organizer details

Events (`/api/events`)

- `GET /events` - List public events
- `GET /events/:id` - Get event details
- `POST /events` - Create event (organizer only)
- `PUT /events/:id` - Update event (organizer only)
- `DELETE /events/:id` - Delete event (organizer only)
- `GET /events/organizer/:organizerId` - Get organizer's events
- `GET /events/category/:categoryId` - Get events by category

Tickets (`/api/tickets`)

- `GET /tickets` - List tickets (with filters)
- `GET /tickets/:ticketId` - Get ticket by ID
- `POST /tickets` - Purchase tickets
- `PUT /tickets/:ticketId` - Update ticket
- `DELETE /tickets/:ticketId` - Cancel ticket
- `GET /tickets/user/:userId` - Get user's tickets

- `GET /tickets/event/:eventId` - Get event tickets
- `PUT /tickets/:ticketId/check-in` - Check in ticket
- `POST /tickets/reactivate` - Reactivate cancelled tickets

RSVP (`/api/rsvp`)

- `GET /rsvp/forms` - List RSVP forms
- `GET /rsvp/forms/:formId` - Get RSVP form details
- `POST /rsvp/forms` - Create RSVP form (organizer only)
- `PUT /rsvp/forms/:formId` - Update RSVP form
- `DELETE /rsvp/forms/:formId` - Delete RSVP form
- `POST /rsvp/forms/:formId/responses` - Submit RSVP response
- `GET /rsvp/forms/:formId/responses` - Get form responses
- `GET /rsvp/forms/:formId/analytics` - Get form analytics
- `POST /rsvp/forms/:formId/messages` - Send messages to respondents

Invitations (`/api/invitations`)

- `GET /invitations` - List invitations (organizer/admin)
- `GET /invitations/:invitationId` - Get invitation details
- `POST /invitations` - Create and send invitations
- `PUT /invitations/:invitationId` - Update invitation
- `DELETE /invitations/:invitationId` - Delete invitation
- `GET /invitations/event/:eventId` - Get event invitations
- `POST /invitations/:invitationId/pay` - Process invitation payment
- `POST /invitations/:invitationId/rsvp` - Respond to invitation

Payments (`/api/payments`)

- `POST /payments/chapa/initialize` - Initialize Chapa payment
- `POST /payments/chapa/verify` - Verify Chapa payment
- `POST /payments/santim/initialize` - Initialize Santimpay payment

- `POST /payments/santim/verify` - Verify Santimpay payment
- `GET /payments/:paymentId` - Get payment details

Withdrawals (`/api/withdrawals`)

- `GET /withdrawals` - List withdrawals (admin)
- `POST /withdrawals` - Request withdrawal (organizer)
- `GET /withdrawals/:id` - Get withdrawal details
- `PUT /withdrawals/:id/status` - Update withdrawal status (admin)
- `GET /withdrawals/organizer/:organizerId` - Get organizer withdrawals
- `GET /withdrawals/organizer/:organizerId/balance` - Get organizer balance (optimized)

Admin (`/api/admin`)

- `GET /admin/dashboard/stats` - Admin dashboard statistics (optimized)
- `GET /admin/users` - Manage users
- `GET /admin/events` - Manage events
- `GET /admin/organizers` - Manage organizers
- `GET /admin/withdrawals` - Manage withdrawals
- `POST /admin/withdrawals/:id/approve` - Approve withdrawal
- `POST /admin/withdrawals/:id/reject` - Reject withdrawal

Categories (`/api/categories`)

- `GET /categories` - List categories
- `POST /categories` - Create category (admin)
- `PUT /categories/:id` - Update category (admin)
- `DELETE /categories/:id` - Delete category (admin)

Organizers (`/api/organizers`)

- `GET /organizers/:id` - Get organizer profile

- `GET /organizers/:id/dashboard` - Organizer dashboard (optimized)
- `PUT /organizers/:id` - Update organizer profile
- `GET /organizers/:id/events` - Get organizer events
- `GET /organizers/:id/analytics` - Get organizer analytics

Notifications (`/api/notifications`)

- `GET /notifications` - List user notifications
- `POST /notifications` - Create notification
- `PUT /notifications/:id/read` - Mark notification as read
- `DELETE /notifications/:id` - Delete notification

Contact (`/api/contact`)

- `POST /contact` - Submit contact form

Webhooks (`/api/webhooks`)

- `POST /webhooks/chapa` - Chapa payment webhook
- `POST /webhooks/santim` - Santimpay payment webhook

Short URLs (`/api/short`)

- `POST /short` - Create short URL
- `GET /short/:shortCode` - Redirect to original URL

QR Tickets (`/api/qr-tickets`)

- `GET /qr-tickets/:ticketId` - Get QR ticket details
- `POST /qr-tickets/validate` - Validate QR code
- `POST /qr-tickets/check-in` - Process check-in

Campaigns (`/api/campaigns`)

- `GET /campaigns` - List campaigns

- `POST /campaigns` - Create campaign (admin)
- `PUT /campaigns/:id` - Update campaign

Campaign Pricing (`/api/campaign-pricing`)

- `GET /campaign-pricing` - List campaign pricing
- `POST /campaign-pricing` - Create campaign pricing tier

Invitation Pricing (`/api/invitation-pricing`)

- `GET /invitation-pricing` - List invitation pricing
- `POST /invitation-pricing` - Create invitation pricing tier

Payment Configuration (`/api/config/payment`)

- `GET /config/payment` - Get payment configuration
- `PUT /config/payment` - Update payment configuration (admin)

Optimized Endpoints

The following endpoints have been optimized for performance with embedded data and compound indexing:

1. `GET /api/admin/dashboard/stats` - Returns all admin statistics in <500ms
 2. `GET /api/users/organizers-stats` - Returns organizers with embedded events and statistics in <800ms
 3. `GET /api/organizers/:id/dashboard` - Returns complete organizer dashboard with events, tickets, balance, and withdrawals in <800ms
 4. `GET /api/withdrawals/organizer/:id/balance` - Returns detailed balance breakdown in <400ms
 5. `GET /api/withdrawals` and `GET /api/withdrawals/organizer/:id` - Include summary statistics for status cards
-

Frontend Routes

Public Routes

- `/` - Landing page with featured and trending events
- `/events/[eventSlug]` - Event detail page
- `/buy-tickets/[eventId]` - Ticket purchase flow
- `/s/[shortCode]` - Short URL redirect
- `/contact` - Contact page
- `/privacy` - Privacy policy
- `/terms` - Terms of service
- `/maintenance` - Maintenance page

Authentication Routes

- `/sign-in` - User sign-in
- `/organizer/sign-in` - Organizer sign-in
- `/organizer/sign-up` - Organizer registration
- `/organizer-registration` - Organizer registration flow
- `/organizer-registration/register` - Registration completion
- `/login` - Legacy login page

User Routes

- `/my-account` - User dashboard
- `/my-account/tickets` - User's tickets
- `/my-account/tickets/[id]` - Specific ticket details
- `/my-account/events` - User's events
- `/my-account/giveaway` - Giveaway page
- `/my-account/wishlist` - User's wishlist

Ticket Routes

- `/ticket/[ticketId]` - Ticket display page
- `/rsvp/[ticketId]` - Ticket RSVP page
- `/rsvp-form/[id]` - RSVP form page
- `/review-form/[id]` - Review form page

Payment Routes

- `/payment/success` - Payment success page
- `/payment/failure` - Payment failure page
- `/payment/cancel` - Payment cancellation page

Admin Routes (`/admin`)

- `/admin` - Admin dashboard
- `/admin/events` - Event management
- `/admin/events/add` - Create event
- `/admin/events/edit/[id]` - Edit event
- `/admin/events/categories` - Category management
- `/admin/events/categories/add` - Create category
- `/admin/events/categories/[id]/edit` - Edit category
- `/admin/tickets` - Ticket management
- `/admin/tickets/add` - Add ticket
- `/admin/tickets/[id]` - Ticket details
- `/admin/users` - User management
- `/admin/users/add` - Add user
- `/admin/users/[id]/edit` - Edit user
- `/admin/organizers` - Organizer management
- `/admin/organizers/[id]` - Organizer details
- `/admin/organizer-registrations` - Organizer registration management

- `/admin/invitations` - Invitation management
- `/admin/invitations/[eventId]` - Event invitations
- `/admin/rsvps` - RSVP management
- `/admin/rsvps/[id]` - RSVP details
- `/admin/rsvps/[id]/responses` - RSVP responses
- `/admin/rsvps/[id]/analytics` - RSVP analytics
- `/admin/rsvps/[id]/messages` - RSVP messages
- `/admin/banners` - Banner management
- `/admin/banners/add` - Add banner
- `/admin/banners/[id]/edit` - Edit banner
- `/admin/withdrawals` - Withdrawal management
- `/admin/login` - Admin login
- `/admin/events/invitation-pricing` - Invitation pricing
- `/admin/events/campaign-pricing` - Campaign pricing

Organizer Routes (`/organizer`)

- `/organizer` - Organizer dashboard
- `/organizer/events` - Event management
- `/organizer/events/create` - Create event
- `/organizer/events/[id]` - Event details
- `/organizer/events/edit/[id]` - Edit event
- `/organizer/account` - Account settings
- `/organizer/customers` - Customer management
- `/organizer/qr-scanner` - QR code scanner
- `/organizer/withdrawals` - Withdrawal management
- `/organizer/invitations` - Invitation management
- `/organizer/notifications` - Notification center
- `/organizer/help` - Help page

- `/organizer/rsvp-builder` - RSVP form builder
- `/organizer/rsvp-builder/[id]` - RSVP form editor
- `/organizer/rsvp-builder/[id]/responses` - RSVP responses
- `/organizer/rsvp-builder/[id]/analytics` - RSVP analytics
- `/organizer/rsvp-builder/[id]/messages` - RSVP messages
- `/organizer/campaign` - Campaign management
- `/organizer/RSVP` - RSVP management

Special Routes

- `/guest-invitation` - Guest invitation page
 - `/guest-invitation/signature` - Guest signature page
-

Key Features

1. Event Management

- **Event Creation:** organizers can create events with details like title, description, date/time, location, and capacity
- **Multi-ticket Types:** Support for multiple ticket types with individual pricing and availability
- **Wave-based Ticket Sales:** Advanced ticket availability management with different switch modes (date, quantity, time, sold-out)
- **Multi-currency Support:** ETB and USD pricing for international attendees
- **Event Categories:** Organized by categories for better discoverability
- **Event Status Management:** Draft, published, cancelled, completed states
- **Featured/Trending Events:** Promotional flags for highlighted events

2. Ticket Sales System

- **Online Ticket Purchase:** Seamless checkout flow with payment gateway integration

- **QR Code Tickets:** Auto-generated QR codes with embedded ticket information and branding
- **Ticket Validation:** QR scanner for check-in and validation
- **Ticket Status Tracking:** Active, used, cancelled, expired, pending states
- **Purchase History:** Complete transaction records for users and organizers
- **Bulk Ticket Operations:** Support for multiple ticket purchases

3. Payment Integration

- **Chapa Payment Gateway:** Ethiopian payment gateway integration
- **Santimpay Payment Gateway:** Alternative Ethiopian payment solution
- **Payment Verification:** Automated webhook-based payment confirmation
- **Multi-currency Processing:** Handle ETB and USD transactions
- **Payment Status Tracking:** Real-time payment status updates
- **Refund Processing:** Support for ticket cancellations and refunds

4. Invitation System

- **Email Invitations:** Send personalized email invitations to guests
- **SMS Invitations:** Send SMS invitations for immediate reach
- **Guest Management:** Track invitation status and responses
- **Paid Invitations:** Support for paid invitation tickets
- **RSVP Tracking:** Monitor guest responses to invitations
- **Invitation Analytics:** Track delivery and response rates

5. RSVP Forms

- **Custom Form Builder:** Create customizable RSVP forms with drag-and-drop interface
- **Multiple Question Types:** Text, choice, dropdown, date, rating, emoji, NPS, file upload
- **Conditional Logic:** Show/hide questions based on previous answers

- **Multi-section Forms:** Organize forms into logical sections
- **Anonymous Responses:** Option for anonymous submissions
- **Payment Integration:** Collect payments with RSVP forms
- **Response Analytics:** Comprehensive analytics and reporting
- **Review Forms:** Separate form type for collecting reviews/feedback

6. Organizer Dashboard

- **Event Overview:** Quick view of all events and their status
- **Ticket Sales Analytics:** Real-time sales data and trends
- **Revenue Tracking:** Complete revenue breakdown by event and ticket type
- **Customer Management:** View and manage customer information
- **Withdrawal Management:** Request and track withdrawal requests
- **QR Scanner:** Built-in QR code scanner for event check-in
- **Notification Center:** Receive important platform notifications

7. Admin Dashboard

- **Platform Overview:** Complete platform statistics and metrics
- **User Management:** Manage users, organizers, and administrators
- **Event Oversight:** Monitor and manage all platform events
- **Revenue Analytics:** Track platform revenue and commissions
- **Withdrawal Processing:** Approve or reject organizer withdrawal requests
- **System Configuration:** Configure payment gateways and platform settings

8. User Experience

- **Responsive Design:** Mobile-first approach for all devices
- **Fast Performance:** Optimized loading times and smooth interactions
- **Real-time Updates:** Socket.IO for live updates and notifications
- **Search and Filtering:** Advanced search for events and tickets

- **Wishlist Functionality:** Save events for later
- **Social Sharing:** Easy sharing of events and tickets

9. Security Features

- **JWT Authentication:** Secure token-based authentication
- **Password Hashing:** Bcrypt for secure password storage
- **CORS Protection:** Configured cross-origin resource sharing
- **Helmet Security:** HTTP security headers
- **Input Validation:** Server-side validation for all inputs
- **Rate Limiting:** Protection against brute force attacks
- **SQL Injection Prevention:** Parameterized queries with Mongoose

10. Analytics and Reporting

- **Event Analytics:** Attendance, sales, and revenue data
 - **RSVP Analytics:** Response rates and form analytics
 - **User Analytics:** User engagement and behavior tracking
 - **Financial Reports:** Revenue breakdown and commission tracking
 - **Export Functionality:** Export data to CSV/Excel formats
-

Development Setup

Prerequisites

- Node.js (v18 or higher)
- MongoDB (local instance or MongoDB Atlas)
- npm or yarn

Environment Variables

Frontend (`.env`)

```
NEXT_PUBLIC_API_URL=http://localhost:5000  
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

Backend (.env)

```
PORT=5000
NODE_ENV=development
MONGODB_URI=mongodb://localhost:27017/pazimo
JWT_SECRET=your_jwt_secret_key
JWT_EXPIRES_IN=30d

# Chapa Payment
CHAPA_SECRET_KEY=your_chapa_secret_key
CHAPA_PUBLIC_KEY=your_chapa_public_key

# Santipay Payment
SANTIMPAY_MERCHANT_ID=your_merchant_id
SANTIMPAY_API_KEY=your_api_key

# Email Configuration
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=your_email@gmail.com
EMAIL_PASS=your_email_password

# SMS Configuration
SMS_API_KEY=your_sms_api_key
SMS_SENDER_ID=PAZIMO

# Cloudinary
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret

# CORS Configuration
FRONTEND_URL=http://localhost:3000
CORS_ORIGINS=http://localhost:3000,https://pazimo.com
```

Installation Steps

1. Clone the repository

```
git clone <repository-url>  
cd pazimo
```

2. Install backend dependencies

```
cd backend  
npm install
```

3. Install frontend dependencies

```
cd ../frontend  
npm install
```

4. Setup MongoDB

- Ensure MongoDB is running locally
- Or configure MongoDB Atlas connection in backend `.env`

5. Configure environment variables

- Copy `.env.example` files if available
- Set up all required environment variables

6. Start the backend server

```
cd backend  
npm run dev
```

7. Start the frontend development server

```
cd frontend  
npm run dev
```

8. Access the application

- Frontend: <http://localhost:3000>
- Backend API: <http://localhost:5000>
- API Health Check: <http://localhost:5000/api/health>

Database Setup

Create Initial Admin User

```
cd backend  
npm run create-admin
```

Create Database Indexes

```
cd backend  
npm run indexes:create
```

Verify Admin User

```
cd backend  
npm run verify-admin
```

Development Scripts

Backend

```
npm start           # Start production server  
npm run dev         # Start development server with nodemon  
npm run create-admin # Create initial admin user  
npm run verify-admin # Verify admin user setup  
npm run indexes:create # Create database indexes  
npm run indexes:sync  # Sync database indexes
```

Frontend

```
npm run dev      # Start development server with turbopack
npm run build    # Build for production
npm start        # Start production server
npm run lint     # Run ESLint
```

Deployment

Frontend Deployment (Vercel)

1. Connect to Vercel

```
npm install -g vercel
vercel login
```

2. Deploy frontend

```
cd frontend
vercel
```

3. Configure environment variables in Vercel dashboard

- NEXT_PUBLIC_API_URL
- NEXT_PUBLIC_APP_URL

Backend Deployment (cPanel/VPS)

Prerequisites

- Node.js hosting (cPanel with Node.js or VPS)
- MongoDB database (MongoDB Atlas recommended)

- SSL certificate

Deployment Steps

1. Upload backend files to server

- Upload all backend files to your hosting directory
- Ensure `.env` file is properly configured

2. Install dependencies

```
cd public_html/backend  
npm install --production
```

3. Configure Node.js application

- In cPanel: Setup Node.js app
- Set application root: `backend`
- Application URL: your domain
- Application startup file: `src/server.js`
- Set environment variables

4. Start the application

- Through cPanel Node.js interface or
- Using PM2: `pm2 start src/server.js --name pazimo-backend`

5. Configure SSL

- Enable SSL certificate for your domain
- Update CORS origins in backend `.env`

MongoDB Atlas Setup

1. Create MongoDB Atlas account
2. Create a new cluster
3. Get connection string

4. **Configure whitelist IP** (0.0.0.0/0 for any IP)
5. **Update backend** `.env` with `MONGODB_URI`

Cloudinary Setup

1. **Create Cloudinary account**
2. **Get API credentials**
3. **Configure environment variables**
4. **Test image upload functionality**

Payment Gateway Setup

Chapa

1. **Create Chapa merchant account**
2. **Get API keys**
3. **Configure webhook URL**
4. **Update environment variables**

Santimpay

1. **Create Santimpay merchant account**
2. **Get merchant credentials**
3. **Configure integration**
4. **Update environment variables**

Monitoring and Maintenance

Health Checks

- Regular health checks: `GET /api/health`
- Monitor server logs for errors
- Track database performance

Database Maintenance

- Regular backup of MongoDB database
- Monitor disk space usage
- Optimize indexes periodically

Log Management

- Configure log rotation
 - Monitor error logs
 - Set up alerts for critical errors
-

Security Considerations

Authentication & Authorization

- **JWT Tokens:** Use secure, short-lived JWT tokens
- **Password Hashing:** All passwords hashed with bcrypt (12 rounds)
- **Role-Based Access:** Strict role-based access control (RBAC)
- **Token Refresh:** Implement token refresh mechanism

Data Protection

- **Environment Variables:** Never commit sensitive data to repository
- **Input Validation:** Server-side validation for all inputs
- **SQL Injection Prevention:** Use parameterized queries (Mongoose)
- **XSS Protection:** Sanitize user inputs, use Content Security Policy

API Security

- **CORS Configuration:** Strict CORS policy for allowed origins
- **Rate Limiting:** Implement rate limiting for API endpoints

- **Helmet Security:** HTTP security headers via Helmet
- **Request Size Limits:** Limit request body size to prevent DoS

Payment Security

- **Webhook Verification:** Verify webhook signatures
- **PCI Compliance:** Follow PCI DSS guidelines for payment processing
- **Secure Data Transmission:** Use HTTPS for all payment-related requests
- **Tokenization:** Use payment gateway tokenization when possible

File Upload Security

- **File Type Validation:** Validate file types and sizes
- **Cloud Storage:** Use Cloudinary for secure file storage
- **Malware Scanning:** Implement virus scanning for uploaded files
- **Access Control:** Restrict file access to authorized users

Database Security

- **MongoDB Security:** Enable MongoDB authentication
- **Network Security:** Use VPC or firewall to restrict database access
- **Backup Encryption:** Encrypt database backups
- **Query Optimization:** Use indexed fields to prevent slow queries

Monitoring & Logging

- **Error Tracking:** Implement error tracking (Sentry, LogRocket)
 - **Access Logs:** Monitor API access patterns
 - **Security Headers:** Implement security headers (CSP, X-Frame-Options)
 - **Regular Audits:** Conduct regular security audits
-

Performance Optimization

Database Optimization

- **Indexing:** Comprehensive indexing strategy for all collections
- **Query Optimization:** Use lean queries and selective field projection
- **Connection Pooling:** Configure MongoDB connection pool
- **Caching:** Implement Redis caching for frequently accessed data

Frontend Optimization

- **Code Splitting:** Automatic code splitting with Next.js
- **Image Optimization:** Use Next.js Image component
- **Lazy Loading:** Implement lazy loading for components and routes
- **Bundle Size:** Monitor and optimize bundle size

API Optimization

- **Response Compression:** Enable gzip compression
- **Pagination:** Implement pagination for large datasets
- **Embedded Data:** Use Mongoose population to reduce API calls
- **CDN:** Use CDN for static assets

Real-time Features

- **Socket.IO:** Efficient real-time communication
 - **Event Optimization:** Minimize socket events and payload size
 - **Reconnection:** Handle socket disconnection gracefully
-

Troubleshooting

Common Issues

Backend won't start

- Check MongoDB connection
- Verify environment variables
- Check port availability
- Review server logs

Frontend build fails

- Clear node_modules and reinstall
- Check TypeScript errors
- Verify environment variables
- Review build logs

Payment failures

- Verify API keys
- Check webhook configuration
- Test payment gateway status
- Review payment logs

Database connection issues

- Verify MongoDB URI
- Check network connectivity
- Ensure MongoDB service is running
- Review connection pool settings

QR code generation fails

- Check logo file path
 - Verify QR code library installation
 - Review ticket data format
 - Check file permissions
-

Future Enhancements

Planned Features

- **Mobile Apps:** Native iOS and Android applications
 - **Advanced Analytics:** Machine learning-powered insights
 - **Social Integration:** Social media sharing and login
 - **Multi-language Support:** Internationalization (i18n)
 - **Advanced Notification System:** Push notifications, email templates
 - **Enhanced Reporting:** Custom report builder
 - **API Rate Limiting:** Advanced rate limiting and throttling
 - **Caching Layer:** Redis implementation for improved performance
 - **Event Recommendations:** Personalized event suggestions
 - **Loyalty Program:** Customer rewards and loyalty points
-

Support and Maintenance

Technical Support

- For technical issues: support@pazimo.com
- Documentation: <https://docs.pazimo.com>
- Status page: <https://status.pazimo.com>

Development Team

- Backend Development: Express.js, MongoDB, Socket.IO
- Frontend Development: Next.js, React, TypeScript
- DevOps: Vercel, MongoDB Atlas, Cloudinary

Contributing

- Fork the repository
 - Create feature branch
 - Follow coding standards
 - Submit pull request
 - Code review process
-

License and Legal

- **License:** Proprietary - All rights reserved
 - **Terms of Service:** <https://pazimo.com/terms>
 - **Privacy Policy:** <https://pazimo.com/privacy>
 - **Contact:** legal@pazimo.com
-

Appendix

API Response Codes

- **200** - Success
- **201** - Created
- **400** - Bad Request
- **401** - Unauthorized

- 403 - Forbidden
- 404 - Not Found
- 500 - Internal Server Error

Error Response Format

```
{
  "status": "error",
  "message": "Error description",
  "stack": "Error stack trace (development only)"
}
```

Success Response Format

```
{
  "status": "success",
  "data": { ... },
  "meta": { ... }
}
```

Document Version: 1.0

Last Updated: June 18, 2026

Maintained By: Pazimo Development Team